Note: Students are required to write the programs along with atleast two outputs for each experiment in the laboratory observation record. Students should take the signature from the faculty in charge. For every laboratory session, the observation component carries **marks** .

Algorithm:

An **algorithm** is simply a step-by-step set of unambiguous instructions designed to solve a specific problem or perform a task.

Think of it like a recipe: you give it specific inputs (ingredients), follow a precise set of steps in order, and end up with a predictable output (a finished dish). In computing, algorithms tell the computer *how* to process data to get the desired result.

Key Properties of an Algorithm

For a sequence of steps to qualify as a proper algorithm, it must satisfy five core properties:

- **Input:** It accepts **zero or more** well-defined inputs from the outside world.
- **Output:** It produces **at least one** defined result or output that answers the problem.
- **Definiteness (Clarity):** Every step must be clear, precise, and completely unambiguous. There should be no room for guesswork.
- **Finiteness:** The algorithm **must terminate** after a finite number of steps. It cannot run in an infinite loop.
- **Effectiveness:** Each instruction must be basic enough to be carried out in practice using finite resources and time.

Example: Finding the Largest Number in a List

Here is a simple algorithm to find the largest value in a group of numbers.

Problem

Find the largest number in this list: **[4, 12, 7, 19, 3]**

Algorithm Steps

1. **Start.**

2. **Take Input:** Accept the list of numbers.
3. **Initialize:** Assume the first number is currently the largest.
    - Largest = 4
4. **Compare and Update (Loop):** Check each remaining number in the list one by one:
    - Check 12: Since $12 > 4$, update Largest = 12.
    - Check 7: Since $7 < 12$, keep Largest = 12.
    - Check 19: Since $19 > 12$, update Largest = 19.
    - Check 3: Since $3 < 19$, keep Largest = 19.
5. **Reach End:** No more numbers left to check.
6. **Output:** Return Largest (which is **19**).
7. **End.**

Problem Statements

1. The Zombie Bridge Crossing (Greedy / Min-Max Optimization)

- **Problem Statement:** Taking an internship at a secluded mountain lab seemed like a great idea until you accidentally pull a lever that unleashes a horde of zombies. You, an Assistant, a Janitor, and an ancient Professor must escape across a rickety bridge at night.
    - The bridge can hold at most 2 people at a time.
    - You have only 1 lantern, which is required for anyone crossing.
    - Crossing times: You (1 minute), Assistant (2 minutes), Janitor (5 minutes), Professor (10 minutes).
    - When two people cross together, they move at the speed of the slower person.
- **Question:** What is the minimum total time required for all 4 people to safely cross the bridge before the zombies reach it?

2. The Counterfeit Gold Coin (Ternary Division / Decision Trees)

- **Problem Statement:** An emperor's treasury contains 12 gold coins, but 1 of the coins is counterfeit and weighs slightly less than the genuine coins. You are given a balance scale that compares the weight of two sets of coins.

- **Question:** What is the minimum number of balance scale weighings needed to guarantee finding the counterfeit coin?

3. The 100 Uncle's Lockers (Number Theory / Perfect Squares)

- **Problem Statement:** An eccentric uncle leaves a row of 100 closed lockers to 100 heirs.
    - Person 1 walks down the row and opens every locker.
    - Person 2 walks down and toggles (closes) every 2nd locker.
    - Person 3 toggles every 3rd locker, and so on, until Person 100 toggles the 100th locker.

- **Question:** Which lockers remain open after all 100 people have finished toggling?

4. The Band Member Instrument Boxes (Cycle Search / Permutation Cycles)

- **Problem Statement:** 10 band members have their musical instruments randomly placed inside 10 closed boxes. Each musician enters the room individually and can open at most 5 boxes to find their assigned instrument before closing all boxes. They cannot communicate once the process begins. If all 10 members find their own instruments, they succeed; if even one fails, everyone loses.

- **Question:** What strategy gives the band a success rate of over 30% instead of random guessing (0.1%)?

5. The Pirate Booty Division (Game Theory / Backward Induction)

- **Problem Statement:** 5 pirates (ranked in seniority 1 to 5) must divide 100 gold coins. The senior-most pirate proposes a distribution plan. All living pirates vote. If at least 50%

approve, the plan is executed; otherwise, the proposer is thrown overboard, and the next senior pirate proposes. All pirates are rational, greedy, and bloodthirsty.

- **Question:** How should Pirate 5 (the most senior) propose the split to maximize gold while surviving?

6. The Airborne Virus Containment (Graph Traversal / Hamiltonian Path)

- **Problem Statement:** An earthquake breaks virus vials in a 4×4 grid of 16 laboratory rooms. You enter at the top-left corner room (Room 1) and must pass through every contaminated room to destroy the virus before escaping through the exit at the bottom-left corner room (Room 13). You cannot re-enter a room once purified.

- **Question:** What path allows you to visit all 16 rooms exactly once and end at the exit room?

7. The Fabergé Egg Drop (Dynamic Programming / Min-Max Drops)

- **Problem Statement:** A museum thief has 2 identical Fabergé eggs and wants to find the highest floor of a 100-story skyscraper from which an egg can be dropped without breaking. If an egg breaks, it cannot be dropped again.

- **Question:** What is the minimum number of drops required in the worst-case scenario to

8. The Wildebeest & Lions River Crossing (State Space Search)

- **Problem Statement:** 3 lions and 3 wildebeest must cross a river using a boat that holds at most 2 animals at a time. At least 1 animal must row the boat back. If lions ever outnumber wildebeest on either bank, the lions will attack the wildebeest.

- **Question:** What sequence of boat trips safely moves all 6 animals across?

9. The Coliseum Guard Passcode (Boolean Logic / Truth Tables)

- **Problem Statement:** Captured prisoners must select the correct escape door out of 3 doors based on clues given by 3 guards.
    - Guard A always tells the truth.
    - Guard B always lies.
    - Guard C answers randomly.

- You do not know which guard is which, and they only answer "Yes" or "No".

- **Question:** What structural logical question guarantees revealing a trustworthy guard to identify the correct door?

10. The Space Monolith Probe (Finite State Automata)

- **Problem Statement:** A research probe must pass through 4 hazardous atmospheric layers surrounding an alien monolith. The probe can be coated with 1 of 3 protective shields: Red, Purple, or Green.
  - Layer 1 degrades Red coatings.
  - Layer 2 degrades Purple coatings.
  - Layer 3 degrades Green coatings.
  - Re-applying a coating takes time, and no two consecutive layers can use the same color.

- **Question:** How can you sequence the shield applications so the probe reaches the monolith undamaged?

****

FLOWCHART

FLOWCHARTS :
A **flowchart** is a visual diagram that represents an algorithm, process, or workflow. It uses standardized geometric shapes connected by arrows to illustrate the step-by-step sequence of operations required to solve a problem or complete a task from start to finish.

Flowcharts break down complex logic into easy-to-read visual maps. To read and create flowcharts effectively, you need to understand their **standard symbols** and **control structures**.

1. Standard Flowchart Symbols

Each shape in a flowchart has a specific meaning:

- **Oval / Pill (Terminator):** Represents the **Start** or **End** of a process.
- **Parallelogram (Input / Output):** Represents receiving data (e.g., *Read User Input*) or displaying results (e.g., *Print Output*).
- **Rectangle (Process):** Represents an action, calculation, or internal operation (e.g., *Calculate Total = Price + Tax*).
- **Diamond (Decision):** Represents a conditional choice or question (e.g., *Is Score ≥ 50?*). It always has two or more branching paths leading out (typically **Yes/No** or **True/False**).
- **Arrows (Flowlines):** Show the direction of execution from one step to the next.

Symbol	Functionality
	Start/end
	Input/output
	Process/computation
	Decision
	Predefined/User-defined function
	Flow indicator
	Connector

Basic Control Structures

Every algorithm can be visualised using three fundamental flow structures:

1. **Sequential Flow:** Steps execute one after another in a straight line from top to bottom.
2. **Conditional Flow (Branching):** The flow splits into different paths based on a decision check (using a Diamond shape).
3. **Iterative Flow (Looping):** A path loops back to a previous step to repeat a process until a specific condition is satisfied.

Why Use Flowcharts?

- **Clarity:** They turn dense, complex written logic into clear, visual steps.
- **Problem-Solving & Debugging:** They help programmers spot logical flaws or missing edge cases before writing code.
- **Communication:** They bridge the gap between technical developers and non-technical stakeholders.

Problem Statements

1. Traffic Signal Driver Action

Problem

Design a flowchart that checks the current color of a traffic light and tells a driver what to do:

- If the light is Red, display "Stop".
- If the light is Yellow, display "Slow Down".
- If the light is Green, display "Go".

2. Exam Pass / Fail Checker

Problem

A school considers a student to have passed an exam if their score is 50 or higher out of 100. Draw a flowchart that reads a student's score and determines if they "Passed" or "Failed".

3. Finding the Larger of Two Numbers

Problem

Create a flowchart that takes two numbers, A and B, compares them, and prints which number is larger. (Assume the two numbers are not equal)

4.Simple Purchase Approval

Problem

An online store needs to check if a customer has enough money to buy an item.

- Input the item's Price and the customer's Account Balance.
- If Balance >= Price, approve the purchase and subtract the price from the balance.
- If Balance < Price, display "Insufficient Funds".\

Question: Draw a flowchart that models the complete logic and decision-making process for this purchase approval system.

5. Self-Checkout Discount & Tax Calculator

Problem

A retail self-checkout counter calculates final bill totals based on customer loyalty status and purchase total (T):

- If $T \geq \$100$, apply a **20% discount**.
- If $T < \$100$ AND customer is a **Store Member**, apply a **10% discount**.
- Otherwise, apply **0% discount**.
- After applying discount, add a **5% sales tax** to calculate the final total F.

Question: Construct a flowchart to calculate final payment amount F.

6. Hospital Patient Triage Classification

Problem

An automated triage workstation reads a patient's **Oxygen Saturation (O_2 in %)** and **Heart Rate (HR in bpm)** to assign medical urgency:

- **Critical:** If $O_2 < 90\%$ OR $HR > 120\ bpm$.

- **Urgent:** If NOT Critical, but $O_2 < 95\%$ OR $HR > 100\ bpm$.
- **Standard:** If $O_2 \geq 95\%$ AND $HR \leq 100\ bpm$.

Question: Draw a flowchart to classify patients into Critical, Urgent, or Standard priority.

7. Smart Thermostat Eco Mode Logic

Problem

A smart home thermostat controls heating/cooling based on a motion sensor and target temperature:

- Check motion sensor every minute.
- If **Motion Detected == True**, set target temperature to user preference T_{user} and reset IdleTimer = 0.
- If **Motion Detected == False**, increment IdleTimer = IdleTimer + 1.
- If IdleTimer >= 30 mins, activate **Eco Mode** (Set target temperature to 24°C).

Question: Create a flowchart showing the loop logic for maintaining Eco Mode vs User Comfort Mode.

8. Parking Garage Entry Barrier Gate

Problem

An automated parking entry system manages available spots (S) out of a maximum capacity:

- When a vehicle approaches the entry sensor:
  - Check if available spots $S > 0$.
  - If $S > 0$, issue a parking ticket, decrement $S = S - 1$, raise barrier gate, wait for car to cross exit sensor, then lower barrier gate.
  - If $S == 0$, turn ON "LOT FULL" display sign and keep barrier gate closed.

Question: Design a flowchart to model the entry gate controller.

Prepared By,
Devna Sumesh